

项目解剖与 Agent 全链路学习手册

目标：把当前项目当作一个可落地的 Agent 工程样本来学习。读完本手册后，应能回答三个问题：

1. 这个项目每一层在做什么；
2. 一次质检任务从 UI / CLI 到工具调用、评分、报告输出的完整链路是什么；
3. 要继续深入 Agent 开发，应按什么顺序学习代码和技术栈。

1. 项目一句话定位

这个项目不是单纯的脚本，也不是只会聊天的大模型应用，而是一个“质检 Agent + 评测工具 + 可视化 UI”的小型工程。

它的业务目标是：给定一批题目数据集，自动判断题目类型，调用评测工具获取模型作答证据，再用规则和可选的大模型归因生成难度判断、风险标签、人工复核队列和质检报告。

当前支持三类数据集：

- **code**：OJ 编程题，走“模型生成 Python 代码 -> Judge0 判题 -> 统计通过率”。
- **math**：判断 / 单选 / 多选等客观题，走“模型多次作答 -> 本地答案标准化 -> 统计正确率”。
- **mixed**：混合题库，先拆成 code/math 两路，再合并结果。

2. 总体架构

项目可以分成三层：

```
apps/quality_ui
  Streamlit UI 层：表单入口、对话入口、报告展示、配置记忆

agents/quality_agent
  Agent 编排层：LangGraph 工作流、LangChain Tool 适配、规则评分、LLM 归因、报告生成

tools/oj_eval
  评测工具层：模型调用、代码判题、客观题判分、评测结果落盘
```

这三层的边界很重要：

- UI 不直接做评测，只收集参数并展示结果。
- Agent 不重写评测逻辑，只负责流程编排、判断和报告。
- **oj_eval** 不理解业务质检结论，只产出可验证的评测证据。

最核心的工程思想是：

大模型负责理解和解释
脚本工具负责执行和验证

Agent 负责调度、状态流转和决策
报告系统负责沉淀证据

3. 三个主要入口

3.1 UI 入口

文件: `apps/quality_ui/quality_ui/app.py`

启动方式:

```
streamlit run apps/quality_ui/quality_ui/app.py
```

或:

```
oj-quality-ui
```

UI 主要有四个业务区:

- 模型配置: 配置 solver 模型、agent 模型、Langfuse。
- 表单质检: 固定工作流入口, 点击后直接跑一次 `run_quality_agent`。
- Agent 会话: 对话式入口, 先判断用户意图, 再决定是否调用质检工具。
- 报告结果: 展示最近一次质检产物。

3.2 Agent CLI 入口

文件: `agents/quality_agent/quality_agent/cli.py`

启动方式:

```
oj-quality-agent run --dataset "cases/mixed/cases_mixed_3_code_3_math.json" --dry-run
```

它做的事情是:

1. 解析命令行参数;
2. 组装 `QualityAgentConfig`;
3. 调用 `run_quality_agent`;
4. 输出本次运行摘要 JSON。

3.3 底层评测工具入口

文件:

- `run_cases_file.py`
- `tools/oj_eval/oj_eval/cli.py`
- `tools/oj_eval/pipeline_app/cli.py`

启动方式：

```
python run_cases_file.py --help
oj-eval --help
```

`run_cases_file.py` 是兼容旧命令的入口，实际会转发到 `oj_eval` 的 CLI。

4. 一次表单质检的完整链路

表单质检是最适合初学者理解全链路的入口。

```
用户在 Streamlit 表单点击“开始质检”
-> app.py 构造 QualityAgentConfig
-> workflow.graph.run_quality_agent()
-> inspect_dataset_node
-> finalize_trace_metadata
-> run_eval_node
-> invoke_oj_eval_tool
-> oj_eval CLI 子进程
-> code/math/mixed 评测
-> score_quality_node
-> 可选 run_llm_analysis_node
-> write_report_node
-> UI 展示 quality_results / review_queue / report_paths
```

对应关键源码：

- `apps/quality_ui/quality_ui/app.py`: `main()`、`_quality_config()`、`_render_quality_state()`
- `agents/quality_agent/quality_agent/workflow/graph.py`: `run_quality_agent()`、`build_graph()`、`_run_sequential()`
- `agents/quality_agent/quality_agent/nodes/inspection.py`: `inspect_dataset_node()`
- `agents/quality_agent/quality_agent/nodes/evaluation.py`: `run_eval_node()`
- `agents/quality_agent/quality_agent/tools/oj_eval_tool.py`: `invoke_oj_eval_tool()`、`run_oj_eval_cli()`
- `agents/quality_agent/quality_agent/nodes/scoring.py`: `score_quality_node()`
- `agents/quality_agent/quality_agent/nodes/analysis.py`: `run_llm_analysis_node()`
- `agents/quality_agent/quality_agent/nodes/reporting.py`: `write_report_node()`

5. Agent 层核心数据结构

5.1 QualityAgentConfig

文件：agents/quality_agent/quality_agent/core/config.py

它表示一次质检任务的运行配置，包括：

- 数据集路径：dataset_path
- 用户要求：user_instruction
- 题库类型：mode
- 每题采样次数：samples
- 输出目录：output_dir
- 是否 dry-run：dry_run
- 是否启用 LLM 归因：enable_llm_analysis
- 是否启用 Langfuse：enable_langfuse
- solver 模型配置：solver_model
- agent 模型配置：agent_model
- Langfuse 配置：langfuse

学习重点：

- 配置对象要集中定义，不要散落在 UI、CLI、节点内部。
- normalized() 用来补默认值、规整路径和类型。
- redacted() 用来输出脱敏版本，避免报告泄露 API Key。

5.2 QualityState

文件：agents/quality_agent/quality_agent/core/state.py

它是 LangGraph 工作流中所有节点共享的状态对象。

主要字段：

- config：本次运行配置。
- run_dir：本次运行目录。
- dataset_type：识别出的题库类型。
- dataset_items：原始题目列表。
- dataset_summary：题库摘要。
- completeness_findings：完整性检查问题。
- eval_summary：评测工具返回的证据。
- quality_results：规则评分后的质检结果。
- llm_findings：Agent 模型归因。
- review_queue：人工复核队列。
- report_paths：报告文件路径。
- trace_metadata：观测系统元数据。
- warnings / errors：不中断流程的异常信息。

学习重点：

- Agent 工程的核心不是“调用一次模型”，而是管理状态如何在节点之间流动。
- TypedDict(total=False) 允许状态逐步补齐，适合工作流节点逐步产出结果。

6. Agent workflow 节点拆解

6.1 数据集识别: `inspect_dataset_node`

文件: `agents/quality_agent/quality_agent/nodes/inspection.py`

职责:

1. 读取 JSON;
2. 兼容 `[...]`、`{"cases": [...]}`、`{"questions": [...]}` 三种外层结构;
3. 按字段识别单题类型;
4. 判断整批数据集是 `code`、`math`、`mixed` 还是 `unknown`;
5. 执行低成本完整性检查。

典型风险:

- 编程题缺 `problem_text`。
- 编程题缺 `stdin` 或 `expected_output`。
- 客观题缺 `question` 或 `answer`。
- 选择题缺 `choices`。
- 题目 ID 重复。
- 题型无法识别。

这是 Agent 的“感知层”: 先用确定性规则理解输入, 不急着调用大模型。

6.2 工具调用: `run_eval_node`

文件: `agents/quality_agent/quality_agent/nodes/evaluation.py`

职责:

1. 读取 `dataset_type`;
2. 调用 `invoke_oj_eval_tool()`;
3. 把评测摘要写回 `eval_summary`;
4. 将工具错误写入 `errors`, 将降级信息写入 `warnings`。

这个节点体现了 Agent 工程中的工具边界: Agent 不直接做题, 不直接判题, 只调用工具拿证据。

6.3 规则评分: `score_quality_node`

文件: `agents/quality_agent/quality_agent/nodes/scoring.py`

职责:

1. 合并完整性检查结果和模型评测结果;
2. 根据通过率生成难度:
 - `>= 0.85`: 简单
 - `>= 0.60`: 中等
 - `>= 0.30`: 困难
 - `> 0`: 超难
 - 有结构错误且通过率为 0: 疑似坏题

3. 生成风险标签；
4. 判断是否需要人工复核。

常见风险标签：

- `low_pass_rate`：通过率偏低
- `suspect_bad_item`：疑似坏题
- `completeness_error`：字段缺失或结构错误
- `completeness_warning`：字段不完整或格式可疑
- `invalid_samples`：存在无效采样
- `excluded_from_accuracy`：未计入正确率
- `dry_run_no_solver_evidence`：Dry run，无真实评测证据

学习重点：

- 规则评分是系统的稳定底座。
- LLM 归因不能替代规则评分，否则容易变成主观判断。

6.4 LLM 归因：run_llm_analysis_node

文件：agents/quality_agent/quality_agent/nodes/analysis.py

职责：

1. 只分析 `need_human_review=True` 的题；
2. 把单题证据压缩成 prompt；
3. 调用 agent 模型；
4. 期望模型返回 `reason`、`risk_reason`、`suggestion`、`confidence`；
5. 失败时只记 warning，不中断主流程。

关键约束：

Only use the provided evidence. Do not invent hidden tests, human reviews, or facts that are not present in the input.

这是防幻觉的关键：模型只能解释已有证据，不能编造不存在的事实。

6.5 报告输出：write_report_node

文件：agents/quality_agent/quality_agent/nodes/reporting.py

职责：

1. 生成 `quality_summary.json`；
2. 生成 `quality_report.md`；
3. 生成 `review_queue.json`；
4. 生成 `completeness_findings.json`；
5. 生成 `trace_metadata.json`。

这些文件分别服务不同对象：

- `quality_report.md`: 给业务和质检人员阅读。
- `quality_summary.json`: 给 UI 和后续程序读取。
- `review_queue.json`: 给人工复核流程使用。
- `completeness_findings.json`: 专门记录结构问题。
- `trace_metadata.json`: 给观测和追踪系统使用。

7. oj_eval 工具层拆解

7.1 代码题链路

核心文件:

- `tools/oj_eval/pipeline_app/application.py`
- `tools/oj_eval/pipeline_app/case_service.py`
- `tools/oj_eval/pipeline_core/runner.py`
- `tools/oj_eval/pipeline_core/services.py`
- `tools/oj_eval/pipeline_core/reporting.py`

完整链路:

```
读取 code JSON
-> 转成 ProblemCase
-> 对每道题扩展 sample_plan
-> 调用 PPIO / OpenAI-compatible chat/completions
-> 从模型回复中提取 Python 代码
-> 提交 Judge0
-> 轮询判题结果
-> 写 sample_summary.json
-> 写 problem_summary.json
-> 写 run_summary.json
```

核心类: `PPIOJudgeRunner`

关键方法:

- `_build_payload()`: 构造模型请求。
- `_call_ppio()`: 调用模型, 兼容 `top_k` 不支持的情况。
- `_parse_response()`: 解析模型输出。
- `run_single_sample()`: 一次采样。
- `run_problem()`: 单题多次采样。
- `run()`: 整批运行。

外部服务封装: `JudgeClient`

- `submit_code()`: 提交代码。
- `get_result()`: 查询判题状态。
- `poll_result()`: 轮询直到完成或超时。
- `judge()`: 提交并轮询的一站式方法。

7.2 客观题链路

核心文件：`tools/oj_eval/pipeline_app/math_quiz.py`

完整链路：

```
读取 math JSON
-> 题型归一化
-> 按题型或 ID 过滤
-> 构造客观题 prompt
-> 模型多次作答
-> 按题型标准化答案
-> 与标准答案比对
-> 统计 pass@k / majority_accuracy
-> 写 run_summary.json
```

核心类：`MathQuizRunner`

关键函数：

- `normalize_question_type()`：题型归一化。
- `normalize_answer_by_type()`：答案归一化。
- `build_math_quiz_prompt()`：构造 prompt。
- `_call_model()`：调用模型，包含 429 指数退避重试。
- `run_single_case()`：单题多次采样。
- `_build_summary()`：生成运行摘要。

7.3 混合题库链路

文件：`agents/quality_agent/quality_agent/tools/oj_eval_tool.py`

当 `dataset_type == "mixed"` 时：

1. 读取原始 JSON；
2. 按字段拆出 `code_items` 和 `math_items`；
3. 写入临时拆分文件；
4. 递归调用 `run_oj_eval_cli()` 分别跑 code/math；
5. 合并 `problems`、`judge_status_summary`、`problem_status_summary`；
6. 返回一个统一的 `eval_summary`。

8. 对话式 Agent 拆解

文件：`agents/quality_agent/quality_agent/workflow/chat_agent.py`

它和表单工作流不同：表单入口一定执行质检；对话入口要先判断用户是不是在要求执行质检。

8.1 普通对话

如果用户只是问候、问能力、追问上一轮结果：

- 不重新运行工具；
- 尝试用 agent 模型回答；
- 如果模型不可用，则用本地摘要回答；
- 保留上一轮 `previous_summary`。

8.2 新质检请求

如果用户明确说“质检、检查、评测、分析数据集、生成报告、重新跑”等：

优先路径：

```
LangGraph ReAct Agent
-> 模型决定是否调用 quality_inspection_tool
-> tool 内部调用固定质量 workflow
-> 模型基于工具结果回复用户
```

兜底路径：

```
如果 ReAct 不可用、模型失败、或者模型没有调用工具
-> 直接执行固定 LangGraph workflow
-> 用本地模板生成中文回复
```

学习重点：

- 真正产品化的 Agent 不能每句话都调用昂贵工具。
- 工具调用失败时要有确定性兜底。
- 多轮会话必须区分“追问上一轮结果”和“重新执行任务”。

9. Agent 记忆机制

这个项目里已经有“记忆”，但它不是向量数据库、RAG 知识库，也不是跨天长期记忆。当前实现更准确地说是三类工程态：

1. UI 会话消息记忆：`agent_chat_messages`
2. 上一轮工具结果摘要记忆：`agent_chat_memory_summary`
3. 落盘证据记忆：`runs/.../quality_summary.json` 等报告文件

理解这三类记忆很关键，因为它们分别解决不同问题：

- `agent_chat_messages` 解决“本轮页面会话里用户和助手刚刚说了什么”。
- `agent_chat_memory_summary` 解决“上一轮质检工具跑出了什么结果，后续追问不要重新跑”。
- `runs/` 目录解决“本次质检证据以后还能不能复盘、给业务看、给程序读”。

9.1 记忆在哪里产生

对话入口在 UI 层产生第一类记忆。

文件：apps/quality_ui/quality_ui/app.py

当用户进入“Agent 会话”页面时，如果 `st.session_state` 里还没有 `agent_chat_messages`，系统会初始化一条助手消息：

请上传 JSON 或使用左侧数据集路径，然后直接输入质检要求。

当用户输入一句话后，UI 会把用户消息追加到会话历史：

```
st.session_state["agent_chat_messages"].append({"role": "user", "content":  
prompt})
```

Agent 生成回复后，再追加助手回复：

```
st.session_state["agent_chat_messages"].append({"role": "assistant", "content":  
response["reply"]})
```

所以第一类记忆的产生位置是 Streamlit 的 `session_state`，不是模型内部。

第二类记忆在工具真实运行后产生。

当 `run_chat_quality_agent()` 返回后，如果本轮确实调用了质检工具：

```
if response.get("tool_was_run") and response.get("summary"):  
    st.session_state["agent_chat_memory_summary"] = response["summary"]
```

也就是说，只有 `tool_was_run=True` 时，才会更新上一轮工具摘要记忆。普通问候、普通追问、配置询问不会覆盖这份工具记忆。

第三类记忆在报告节点产生。

文件：agents/quality_agent/quality_agent/nodes/reporting.py

`write_report_node()` 会把完整运行状态写入本次 `run_dir`：

- quality_summary.json
- quality_report.md
- review_queue.json
- completeness_findings.json
- trace_metadata.json

这些文件才是更长期、可复盘的证据记忆。

9.2 运行中记忆怎么传给 Agent

用户每次在对话框输入后，UI 调用：

```
run_chat_quality_agent(  
    user_message=prompt,  
    config=config,  
    chat_history=st.session_state.get("agent_chat_messages", []),  
    previous_summary=st.session_state.get("agent_chat_memory_summary"),  
    prefer_langgraph=use_langgraph,  
    on_event=on_event,  
)
```

这里有两个关键入参：

- `chat_history`：完整的本页会话消息列表。
- `previous_summary`：上一轮工具运行后的压缩摘要。

进入 `chat_agent.py` 后，`_history_context()` 会把它们压缩成模型可读上下文：

上一轮工具结果摘要如下，回答追问时优先使用这份记忆，不要编造：
{...}

以下是本轮之前的会话历史，回答时可参考，但仍以工具结果为准：
user: ...
assistant: ...

当前实现做了两层压缩：

1. `previous_summary` 只保留关键字段，例如数据集类型、题目数、复核数、报告路径、警告、错误。
2. `chat_history` 只取最近 8 条，并且每条内容截断，避免 prompt 太长。

所以模型看到的不是无限长历史，而是一份经过裁剪的短期上下文。

9.3 Agent 如何判断是追问记忆还是重新运行

文件：`agents/quality_agent/quality_agent/workflow/chat_agent.py`

关键函数：`_looks_like_new_inspection_request()`

逻辑大致是：

如果已经有上一轮摘要，并且用户说的是：

刚才 / 上次 / 上一轮 / 之前 / 结果 / 报告在哪 / 多少 / 为什么 / 解释一下

同时没有说：

重新 / 再跑 / 重跑 / 重新质检 / 重新检查 / 重新评测 / 重新运行

那么系统认为这是追问上一轮结果，不重新执行工具。

如果用户说的是：

质检 / 检查 / 评测 / 数据集 / 题目 / 难度 / 正确率 / 生成报告 / 重新

系统才认为这是新的质检请求，进入工具调用链路。

这就是当前项目的“记忆使用策略”：

追问上一轮 -> 使用 `previous_summary` 回答
明确重新跑 -> 调用质检工具生成新 `summary`
普通聊天 -> 不跑工具，使用会话上下文或本地模板回答

9.4 ReAct Agent 中的临时工具记忆

当配置了 agent 模型且用户明确要求质检时，系统优先使用 ReAct Agent。

文件：`agents/quality_agent/quality_agent/workflow/chat_agent.py`

`_build_quality_tool()` 内部有一个 `tool_runtime` 字典：

```
tool_runtime: Dict[str, Any] = {}
```

当模型决定调用 `quality_inspection_tool` 后，工具会执行固定质检工作流，并把结果塞进：

```
tool_runtime["summary"] = summary
```

这是一种“本次函数调用内的临时记忆”，作用是：

1. 工具结果以 JSON 字符串返回给 ReAct 模型；
2. Python 外层也能从 `tool_runtime["summary"]` 拿到结构化结果；
3. UI 最后把这个 `summary` 存入 `agent_chat_memory_summary`。

它不会自动跨请求存在，只在本次 `run_chat_quality_agent()` 调用期间有效。

9.5 工作流内部状态是不是记忆

是，但它属于“单次运行内记忆”。

文件：`agents/quality_agent/quality_agent/core/state.py`

QualityState 记录一次 workflow 从开始到结束的所有中间结果：

- `dataset_items`
- `dataset_summary`
- `completeness_findings`
- `eval_summary`
- `quality_results`
- `llm_findings`
- `review_queue`
- `report_paths`
- `warnings`
- `errors`

这些字段在 LangGraph 节点之间传递。它们在单次运行中一直存在，直到 `write_report_node()` 把关键内容写入文件。

因此它的生命周期是：

```
run_quality_agent() 开始
-> 初始化 QualityState
-> 每个节点不断补字段
-> write_report_node() 落盘
-> 返回给 UI / CLI
-> UI 可能存在 session_state 里
```

如果不被 UI 保存，也不看落盘文件，那么函数返回后这份内存态就结束了。

9.6 结束后记忆存在哪里

当前项目结束后有两类存储位置。

第一类：浏览器页面会话内存。

位置：

```
st.session_state["agent_chat_messages"]
st.session_state["agent_chat_memory_summary"]
st.session_state["chat_agent_result"]
st.session_state["quality_state"]
```

特点：

- 只在当前 Streamlit 会话中保存；
- 刷新、重启服务、清空会话后可能丢失；
- 适合短期多轮对话；
- 不适合作为正式审计证据。

第二类：本地磁盘报告。

位置一般是：

```
runs/quality_agent/run_quality_YYYYMMDD_HHMMSS_XXXXXX/
```

里面包括：

```
quality_summary.json
quality_report.md
review_queue.json
completeness_findings.json
trace_metadata.json
solver_runs/
```

特点：

- 重启 UI 后仍然存在；
- 可以交给业务或质检人员；
- 可以被后续程序读取；
- 是当前项目最接近“长期记忆”的部分。

9.7 清空会话会清掉什么

UI 中点击“清空会话”时，会执行：

```
st.session_state["agent_chat_messages"] = []
st.session_state.pop("chat_agent_result", None)
st.session_state.pop("agent_chat_memory_summary", None)
st.rerun()
```

它会清掉：

- 对话消息；
- 最近一次对话 Agent 结果；
- 上一轮工具摘要记忆。

它不会删除：

- `runs/` 目录下已经生成的报告；
- `quality_summary.json`；
- `review_queue.json`；
- `solver_runs/` 中的模型请求、响应、判题结果。

所以清空会话只是清掉 UI 会话态，不是删除历史质检证据。

9.8 当前记忆机制的局限

当前实现是 demo 级、工程实用型记忆，优点是简单、可控、容易解释；局限也很明确：

- 没有数据库，不能跨用户管理历史任务。
- 没有向量检索，不能从大量历史报告中语义搜索相似问题。
- 没有长期用户画像，不能记住某个用户的长期偏好。
- 没有自动加载上一次磁盘报告作为新会话记忆。
- `agent_chat_memory_summary` 只保存上一轮工具摘要，不保存多轮工具运行历史。
- `chat_history` 只传最近 8 条，长对话会被裁剪。

这些不是 bug，而是当前阶段的工程取舍。

9.9 如果要升级成真正长期记忆

可以按三层升级。

第一层：任务历史库。

把每次运行的摘要写入 SQLite 或 PostgreSQL：

```
run_id
dataset_path
dataset_hash
dataset_type
created_at
quality_summary_path
review_queue_count
warnings
errors
```

这样 UI 重启后也能列出历史任务。

第二层：语义检索记忆。

把题目、风险原因、人工复核结论、报告摘要做 embedding，写入向量库。

用途：

- 查找相似坏题；
- 复用过往归因；
- 比较同一供应商不同批次数据质量；
- 回答“上次类似问题怎么处理的”。

第三层：人工反馈闭环。

把人工复核结果重新写回系统：

```
agent_difficulty
human_final_difficulty
agent_risk_flags
human_review_result
```

```
accepted_or_rejected
reviewer_comment
```

这样后续才能校准规则阈值、prompt 和模型选择。

当前项目已经有 `review_queue.json`，这是做人工反馈闭环的入口，但还没有实现“人工反馈回写和学习”。

10. LangGraph / LangChain 在项目里的位置

10.1 LangGraph

文件： `agents/quality_agent/quality_agent/workflow/graph.py`

用途：

- 定义节点；
- 定义节点顺序；
- 在 `score_quality` 后做条件路由；
- 如果没有安装 LangGraph，则使用 `_run_sequential()` 顺序兜底。

当前图结构：

```
START
-> inspect_dataset
-> finalize_trace_metadata
-> run_eval
-> score_quality
-> llm_analysis 或 write_report
-> write_report
-> END
```

条件路由：

```
如果 enable_llm_analysis=True 且存在 need_human_review=True 的题
-> llm_analysis
否则
-> write_report
```

10.2 LangChain Tool

文件： `agents/quality_agent/quality_agent/tools/oj_eval_tool.py`

用途：

- 把现有 CLI 封装成 `StructuredTool`；
- 让 Agent 以工具形式调用 `oj_eval`；
- 缺少 LangChain 依赖时直接调用 Python 函数兜底。

学习重点：

- Tool 的输入要结构化。
- Tool 的输出要可被程序继续消费，不能只返回自然语言。
- Tool 调用要记录命令、stdout、stderr、returncode 和脱敏参数。

10.3 ReAct Agent

文件：agents/quality_agent/quality_agent/workflow/chat_agent.py

用途：

- 让模型根据用户消息决定是否调用 `quality_inspection_tool`；
- 如果工具没被调用但用户确实要求质检，则补跑固定工作流。

这个设计避免了一个常见问题：ReAct 模型有时会“只回答不调用工具”。项目里通过兜底逻辑保证最终有可验证结果。

11. Langfuse 可观测性

文件：agents/quality_agent/quality_agent/integrations/langfuse.py

用途：

- 生成 trace metadata；
- 在配置完整时给 LangChain LLM 调用附加 callback；
- 未启用、未安装或未配置密钥时静默降级。

它记录的关键元数据包括：

- 数据集路径；
- 用户指令；
- 数据集类型；
- 采样次数；
- solver 模型；
- agent 模型；
- run_dir。

学习重点：

- Agent 工程必须能追踪输入、输出、耗时和错误。
- 可观测性不能成为主流程的硬依赖，失败时应降级。

12. 当前系统已经具备的能力

12.1 已经比较完整的部分

- 支持 UI、CLI、对话三种入口。
- 支持 code/math/mixed 三类题库。
- 支持 dry-run，便于低成本检查结构。

- 支持多次采样，用通过率反推难度。
- 支持 Judge0 判题。
- 支持客观题答案归一化。
- 支持规则评分和风险标签。
- 支持人工复核队列。
- 支持报告落盘。
- 支持 LangGraph 编排和顺序兜底。
- 支持 LangChain Tool 包装。
- 支持可选 LLM 归因。
- 支持可选 Langfuse 追踪。

12.2 仍然偏 demo / 内部验证的部分

- 没有正式测试目录。
- 数据集 schema 还没有强约束，例如 Pydantic 模型或 JSON Schema。
- 编程题评测依赖提供的 `stdin` / `expected_output`，隐藏测试不足时难度判断会偏弱。
- Agent 归因 prompt 还比较轻量，没有形成严格结构化输出约束。
- 复杂代码题遇到推理型模型只返回 `reasoning_content` 时，可能拿不到可执行代码。
- 本地 UI 支持明文保存密钥，生产环境应改为环境变量或密钥管理。
- Langfuse 云端 trace 需要真实 key 才能完整验证。

13. 技术栈学习路线

阶段 1：先懂 Python 工程结构

学习目标：

- 会从 `pyproject.toml` 看入口命令；
- 会分辨 package、module、CLI、脚本；
- 会理解 dataclass、TypedDict、Path、subprocess；
- 会理解 JSON 读写和日志落盘。

建议阅读顺序：

1. `pyproject.toml`
2. `run_cases_file.py`
3. `tools/oj_eval/pipeline_app/cli.py`
4. `tools/oj_eval/pipeline_app/application.py`

阶段 2：读懂底层评测工具

学习目标：

- 明白 code 题如何生成代码并判题；
- 明白 math 题如何作答、标准化和判分；
- 明白多次采样和通过率如何产生。

建议阅读顺序：

1. `tools/oj_eval/pipeline_core/core.py`
2. `tools/oj_eval/pipeline_core/runner.py`
3. `tools/oj_eval/pipeline_core/services.py`
4. `tools/oj_eval/pipeline_app/math_quiz.py`
5. `tools/oj_eval/pipeline_core/reporting.py`

阶段 3：读懂 Agent workflow

学习目标：

- 理解配置对象；
- 理解状态对象；
- 理解每个节点的输入输出；
- 理解 LangGraph 条件路由；
- 理解顺序兜底。

建议阅读顺序：

1. `agents/quality_agent/quality_agent/core/config.py`
2. `agents/quality_agent/quality_agent/core/state.py`
3. `agents/quality_agent/quality_agent/workflow/graph.py`
4. `agents/quality_agent/quality_agent/nodes/inspection.py`
5. `agents/quality_agent/quality_agent/nodes/evaluation.py`
6. `agents/quality_agent/quality_agent/nodes/scoring.py`
7. `agents/quality_agent/quality_agent/nodes/analysis.py`
8. `agents/quality_agent/quality_agent/nodes/reporting.py`

阶段 4：读懂工具封装

学习目标：

- 明白为什么 Agent 不直接 import 底层 runner；
- 明白 CLI 子进程封装的利弊；
- 明白 LangChain `StructuredTool` 的价值；
- 明白 `stdout/stderr/returncode` 为什么要保存。

建议阅读：

1. `agents/quality_agent/quality_agent/tools/oj_eval_tool.py`

重点函数：

- `run_oj_eval_cli()`
- `invoke_oj_eval_tool()`
- `build_oj_eval_structured_tool()`
- `_merge_mixed_results()`

阶段 5：读懂对话式 Agent

学习目标：

- 区分普通对话、追问、重新质检；
- 理解 ReAct Agent 的工具调用；
- 理解工具未调用时为什么要补跑固定 workflow；
- 理解对话记忆如何压缩。

建议阅读：

1. `agents/quality_agent/quality_agent/workflow/chat_agent.py`

重点函数：

- `run_chat_quality_agent()`
- `_looks_like_new_inspection_request()`
- `_build_quality_tool()`
- `_compact_state()`
- `_format_fallback_reply()`

阶段 6：专门读懂 Agent 记忆

学习目标：

- 明白当前项目的记忆不是向量库，而是 `session_state`、上一轮 summary 和落盘报告。
- 明白什么时候使用上一轮摘要，什么时候重新调用工具。
- 明白 `QualityState`、`agent_chat_messages`、`agent_chat_memory_summary` 的生命周期差异。
- 明白清空会话不会删除 `runs/` 下的历史报告。

建议阅读：

1. `apps/quality_ui/quality_ui/app.py` 中 `agent_chat_messages`、`agent_chat_memory_summary` 相关逻辑
2. `agents/quality_agent/quality_agent/workflow/chat_agent.py` 中 `_history_context()`、`_looks_like_new_inspection_request()`、`_compact_state()`
3. `agents/quality_agent/quality_agent/nodes/reporting.py` 中 `write_report_node()`
4. `agents/quality_agent/quality_agent/core/state.py`

阶段 7：读懂 UI 产品化

学习目标：

- 理解 Streamlit 如何收集参数；
- 理解 `session_state` 如何保存运行状态；
- 理解实时事件如何展示；
- 理解报告文件如何读取和渲染。

建议阅读：

1. `apps/quality_ui/quality_ui/app.py`
2. `apps/quality_ui/quality_ui/config_store.py`
3. `apps/quality_ui/quality_ui/launcher.py`

阶段 8：补齐生产化知识

后续应重点学习：

- Pydantic / JSON Schema：数据集 schema 校验。
- pytest：自动化测试和验收用例。
- LangGraph：复杂条件路由、循环、自修复、人工确认节点。
- LangChain Tool：工具入参、输出 schema、异常处理。
- Langfuse：trace、span、prompt 版本、成本和 latency 监控。
- Judge0：自建服务、沙箱安全、并发、超时、资源限制。
- LLM 评测：pass@k、majority vote、模型阶梯、置信度校准。
- Prompt engineering：结构化输出、防幻觉、证据约束。

14. 建议的实操学习任务

为了真正啃透项目，不建议只读源码。建议按下面任务逐个动手：

任务 1：跑通 dry-run

命令：

```
oj-quality-agent run --dataset "cases/mixed/cases_mixed_3_code_3_math.json" --dry-run
```

观察：

- dataset_type
- dataset_summary
- completeness_findings
- quality_results
- quality_report.md

目标：理解不调用模型时系统能做哪些结构检查。

任务 2：跑通客观题真实评测

命令：

```
python run_cases_file.py --math --math-cases-file  
"cases/math/cases_math_small.json" --math-samples 1
```

观察：

- math_quiz_results.json
- run_summary.json
- sample_result.json

目标：理解客观题的 prompt、答案归一化和判分。

任务 3：跑通代码题真实评测

命令：

```
python run_cases_file.py --domain code --cases-file  
"cases/code/cases_to_test.json" --samples 1
```

观察：

- `ppio_request.original.json`
- `ppio_response.json`
- `generated_code.py`
- `judge_result.json`
- `problem_summary.json`

目标：理解模型生成代码和 Judge0 判题的证据链。

任务 4：手动制造一个坏数据

做法：

- 复制一个小 JSON；
- 删除某道客观题的 `answer`；
- 删除某道代码题的 `expected_output`；
- 重复一个 ID；
- 再跑 dry-run。

观察：

- `completeness_findings.json`
- `risk_flags`
- `review_queue.json`

目标：理解完整性检查如何进入复核队列。

任务 5：打开 Agent LLM 归因

前提：

- 配置 agent 模型；
- 关闭 dry-run 或制造需要复核的题；
- 启用 `enable_llm_analysis`。

观察：

- `llm_findings`
- `review_queue` 中的 `llm_analysis`
- 报告中的风险解释是否只基于已有证据。

目标：理解“LLM 解释证据，而不是代替证据”。

任务 6：用对话入口测试多轮语义和记忆

在 UI 的 Agent 会话里依次输入：

你好
检查这份数据集并生成报告
刚才有多少题需要复核
重新跑一次

观察：

- 哪些轮次调用工具；
- 哪些轮次只回答上一轮摘要；
- `tool_was_run` 的变化；
- 运行过程展示。

目标：理解真实 Agent 产品中的意图识别和工具调用控制。

重点观察：

- 你好 不应该调用工具；
- 检查这份数据集并生成报告 应该调用工具，并更新 `agent_chat_memory_summary`；
- 刚才有多少题需要复核 应该使用上一轮摘要，不重新跑；
- 重新跑一次 应该再次调用工具，并生成新的运行目录；
- 点击“清空会话”后，对话态和上一轮摘要消失，但 `runs/` 下的报告仍然存在。

15. 下一步深入建议

第一轮建议按这个顺序推进：

1. 先画出 `QualityState` 在每个节点前后的字段变化。
2. 再分别跑一遍 dry-run、math、code、mixed。
3. 对照 `runs/` 目录中的输出文件看证据链。
4. 最后再看 UI 和对话入口。

第二轮再做改造：

1. 给数据集输入增加 Pydantic schema。
2. 给 `score_quality()` 增加单元测试。
3. 给 `run_chat_quality_agent()` 增加意图识别测试。
4. 把 Agent 归因输出改成强结构化 JSON。
5. 增加一个新的质检节点，例如“标签一致性检查”。

第三轮再做生产化：

1. 引入真实测试集；
2. 做模型阶梯评测；
3. 接 Langfuse 真 trace；
4. 接人工复核页面；

5. 把报告输出接到采购验收流程。

16. 最重要的理解

这个项目最值得学习的不是某一个库，而是工程分工：

- `oj_eval` 提供可验证证据；
- `quality_agent` 编排流程并做判断；
- `quality_ui` 让人能操作和复核；
- `LangGraph` 管状态流；
- `LangChain Tool` 管工具边界；
- `Langfuse` 管可观测性；
- `Judge0` 管代码执行验证；
- 大模型只在需要理解、生成、解释的地方出现。

如果把所有事情都交给大模型口头判断，就会失去可追溯性；如果只保留脚本，又缺少灵活调度、归因解释和人机协作能力。这个项目的价值正在于把两者结合起来。